# Neural Networks for Dynamical Systems

## MATH797/BDA835

Dr. Xuping Xie

Department of Mathematics & Statistics
Old Dominion University

# Outline


1. Introduction to Dynamical Systems
2. Chaotic Dynamical Systems
3. Hamiltonian Dynamical Systems
4. Numerical Methods for Simulation
5. Neural Networks for Predicting Dynamics
6. Summary

# Introduction to Dynamical Systems

# What is a Dynamical System?

A **dynamical system** is a mathematical description of how a state evolves over time according to a deterministic rule.

**Continuous-time (Ordinary Differential Equation):**

$$\frac{d\boldsymbol{x}}{dt} = \boldsymbol{f}(\boldsymbol{x}, t), \quad \boldsymbol{x}(0) = \boldsymbol{x}_0 \tag{1}$$

**Discrete-time (Iterated Map):**

$$\boldsymbol{x}_{k+1} = \boldsymbol{F}(\boldsymbol{x}_k), \quad k = 0, 1, 2, \ldots \tag{2}$$

**Key components:**

- $\boldsymbol{x}(t) \in \mathbb{R}^n$ — state vector (position in phase space)
- $\boldsymbol{f} : \mathbb{R}^n \times \mathbb{R} \to \mathbb{R}^n$ — vector field (the "rule")
- $\boldsymbol{x}_0$ — initial condition

# Autonomous vs. Non-Autonomous Systems

**Autonomous system:** The rule does not depend explicitly on time

$$\frac{d\boldsymbol{x}}{dt} = \boldsymbol{f}(\boldsymbol{x}) \tag{3}$$

The vector field is "frozen" — same rule applies at all times.

**Non-autonomous system:** Explicit time dependence

$$\frac{d\boldsymbol{x}}{dt} = \boldsymbol{f}(\boldsymbol{x}, t) \tag{4}$$

Example: Driven oscillator with time-varying forcing.

**Trick:** Convert non-autonomous to autonomous by adding time as a state variable:

$$\frac{d}{dt}\begin{pmatrix}\boldsymbol{x}\\ \tau\end{pmatrix} = \begin{pmatrix}\boldsymbol{f}(\boldsymbol{x}, \tau)\\ 1\end{pmatrix} \tag{5}$$

# Phase Space and Trajectories

**Phase space:** The space $\mathbb{R}^n$ of all possible states
**Trajectory (orbit):** The curve $\boldsymbol{x}(t)$ traced by the system starting from $\boldsymbol{x}_0$

**Flow map:** $\phi_t : \mathbb{R}^n \to \mathbb{R}^n$ evolves initial conditions forward in time

$$\boldsymbol{x}(t) = \phi_t(\boldsymbol{x}_0) \tag{6}$$

**Fundamental properties:**

- $\phi_0(\boldsymbol{x}) = \boldsymbol{x}$ (identity at $t = 0$)
- $\phi_{t+s} = \phi_t \circ \phi_s$ (semigroup property)
- Trajectories cannot cross (uniqueness of solutions)

**Implication:** The entire future (and past) is determined by the current state!

# Fixed Points (Equilibria)

**Fixed point:** A state $\boldsymbol{x}^*$ where the system stays forever

$$\boldsymbol{f}(\boldsymbol{x}^*) = \boldsymbol{0} \tag{7}$$

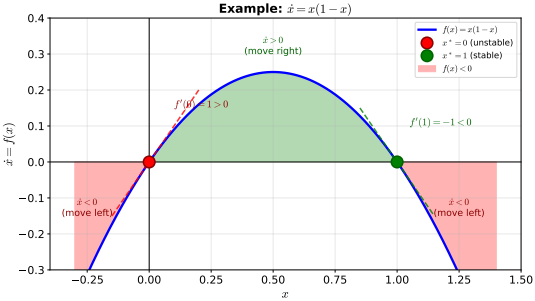
**Finding fixed points:** Solve the algebraic equation $\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{0}$

**Example: 1D system** $\dot{x} = x(1 - x)$

- Fixed points: $x^* = 0$ and $x^* = 1$
- For $0 < x < 1$: $\dot{x} > 0$ (moving right)
- For $x > 1$: $\dot{x} < 0$ (moving left)
- Conclusion: $x^* = 0$ is unstable, $x^* = 1$ is stable

# Stability of Fixed Points: Geometric View

**Example:** $\dot{x} = x(1-x)$




Phase Line: arrows show direction of flow

**1D Stability Criterion:** $x^*$ is stable if $f'(x^*) < 0$, unstable if $f'(x^*) > 0$

# Linear Stability Analysis

**Question:** Is a fixed point $\boldsymbol{x}^*$ stable or unstable?
**Method:** Linearize around $\boldsymbol{x}^*$. Let $\boldsymbol{\xi} = \boldsymbol{x} - \boldsymbol{x}^*$ (small perturbation)

$$\frac{d\boldsymbol{\xi}}{dt} = \boldsymbol{J}\boldsymbol{\xi} + \mathcal{O}(\|\boldsymbol{\xi}\|^2) \tag{8}$$

where $\boldsymbol{J}$ is the **Jacobian matrix**:

$$J_{ij} = \left.\frac{\partial f_i}{\partial x_j}\right|_{\boldsymbol{x}^*} \tag{9}$$

**Stability determined by eigenvalues $\lambda_i$ of $\boldsymbol{J}$:**

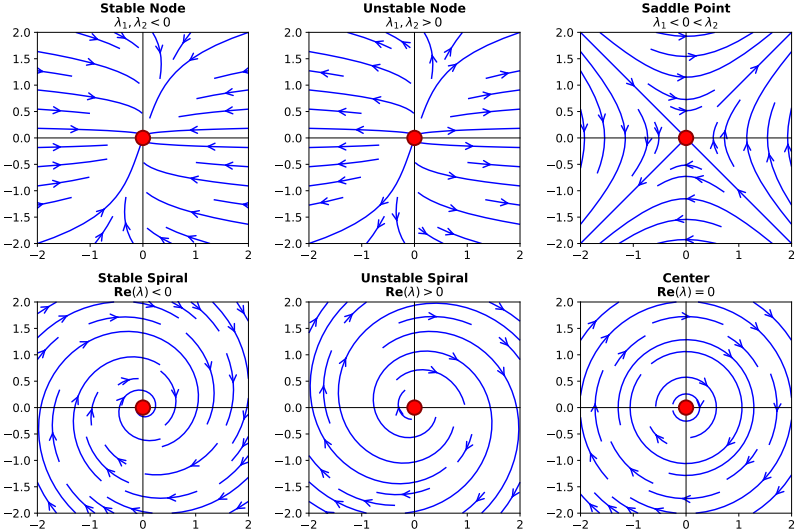
- All $\mathrm{Re}(\lambda_i) < 0 \Rightarrow$ **Stable** (perturbations decay)
- Any $\mathrm{Re}(\lambda_i) > 0 \Rightarrow$ **Unstable** (perturbations grow)
- All $\mathrm{Re}(\lambda_i) = 0 \Rightarrow$ **Center** (need nonlinear analysis)

# Classification of Fixed Points in 2D

For a 2D system with Jacobian eigenvalues $\lambda_1, \lambda_2$:

| **Eigenvalues** | **Type** | **Behavior** |
|---|---|---|
| $\lambda_1, \lambda_2 < 0$ (real) | Stable node | Trajectories converge |
| $\lambda_1, \lambda_2 > 0$ (real) | Unstable node | Trajectories diverge |
| $\lambda_1 < 0 < \lambda_2$ (real) | Saddle point | Converge/diverge along eigenvectors |
| $\mathrm{Re}(\lambda) < 0$, $\mathrm{Im}(\lambda) \neq 0$ | Stable spiral | Spiral inward |
| $\mathrm{Re}(\lambda) > 0$, $\mathrm{Im}(\lambda) \neq 0$ | Unstable spiral | Spiral outward |
| $\mathrm{Re}(\lambda) = 0$, $\mathrm{Im}(\lambda) \neq 0$ | Center | Closed orbits (linear) |

# Phase Portraits of 2D Fixed point

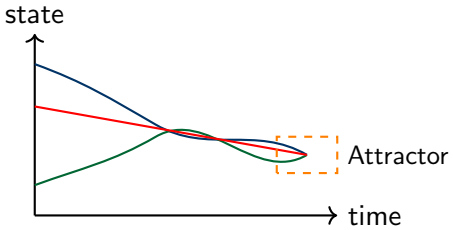


# What is an Attractor? Physical Intuition

**Question:** Where does a dynamical system "end up" after a long time?

**Examples from everyday life:**

- **Pendulum with friction:** Eventually stops at bottom $\rightarrow$ *fixed point*
- **Beating heart:** Repeats the same cycle $\rightarrow$ *limit cycle*
- **Weather:** Never exactly repeats, but stays bounded $\rightarrow$ *strange attractor*




Different initial conditions $\rightarrow$ same long-term behavior

# Attractors

**Attractor:** A set $A$ in phase space that:

1. Is invariant: trajectories starting in $A$ stay in $A$
2. Attracts nearby trajectories: points near $A$ converge to $A$
3. Is minimal: no proper subset has these properties

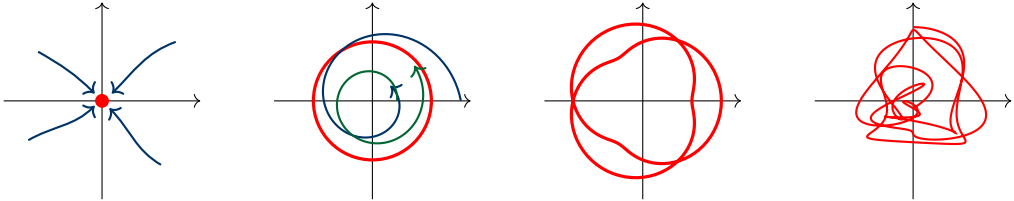
**Types of attractors:**

- **Fixed point:** Zero-dimensional (equilibrium)
- **Limit cycle:** One-dimensional (periodic orbit)
- **Torus:** Two-dimensional (quasi-periodic motion)
- **Strange attractor:** Fractal dimension (chaotic motion)

**Basin of attraction:** Set of initial conditions that converge to attractor $A$

$$\mathcal{B}(A) = \{\boldsymbol{x}_0 : \lim_{t\to\infty} \mathrm{dist}(\phi_t(\boldsymbol{x}_0), A) = 0\} \tag{10}$$

# Attractor Types: Visual Overview



**Fixed Point**
dim = 0

**Limit Cycle**
dim = 1

**Torus**
dim = 2

**Strange Attractor**
dim = fractal

**Physical intuition:**

- Attractors are where the system "settles down"
- Energy dissipation → trajectories converge
- Different systems → different attractor shapes

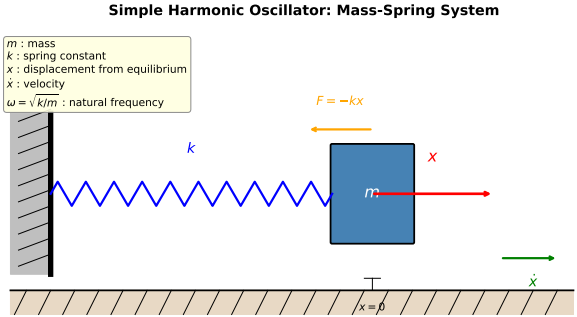
**Key property:**

- Attractor dimension tells us the system's long-term complexity
- Strange attractors: bounded but never repeat!

# Example 1: Simple Harmonic Oscillator

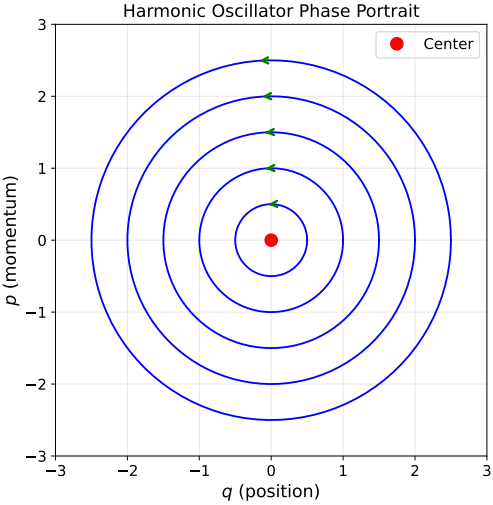
**Physical system:** Mass on a spring (no friction)
**Equation:** $m\ddot{x} + kx = 0$ or $\ddot{x} + \omega^2 x = 0$ where $\omega = \sqrt{k/m}$
**As a first-order system:** Let $x_1 = x$, $x_2 = \dot{x}$

$$\frac{d}{dt}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} x_2 \\ -\omega^2 x_1 \end{pmatrix} \tag{11}$$




**Fixed point:** $(0, 0)$, **Jacobian:** $\boldsymbol{J} = \begin{pmatrix} 0 & 1 \\ -\omega^2 & 0 \end{pmatrix}$
**Eigenvalues:** $\lambda = \pm i\omega$ (purely imaginary)
**Behavior:** Center — closed elliptical orbits (energy conserved!)

# Phase Portrait: Simple Harmonic Oscillator




**Observations:**

- Closed circular orbits
- Origin is a **center**
- Clockwise motion
- Each orbit = constant energy level

**Energy (conserved):**

$$E = \frac{1}{2}p^2 + \frac{1}{2}q^2 = \text{const}$$

Orbits are level sets of $E$.

# Example 2: Simple Pendulum

**Equation of motion:**

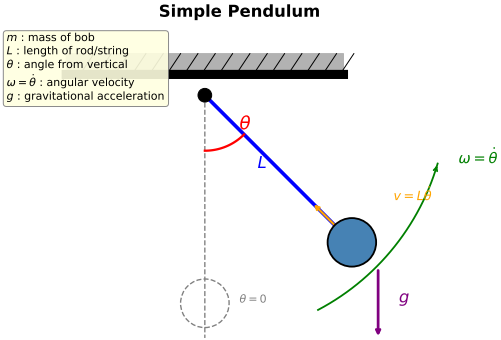
$$\ddot{\theta} + \frac{g}{L}\sin\theta = 0$$

**As a first-order system:**

$$\dot{\theta} = \omega, \quad \dot{\omega} = -\frac{g}{L}\sin\theta$$

**Fixed points:**

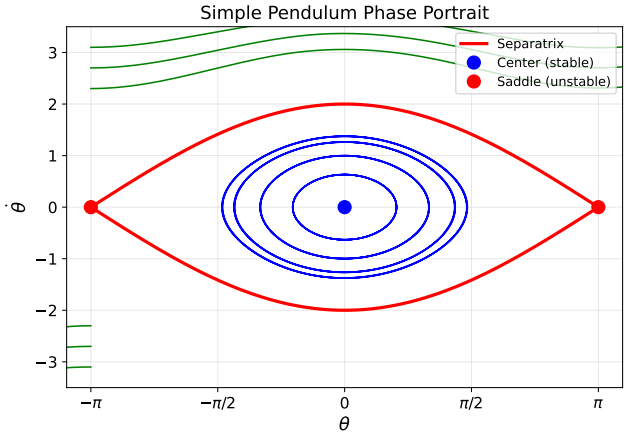
- $(\theta, \omega) = (0, 0)$ — stable center
- $(\theta, \omega) = (\pi, 0)$ — unstable saddle




**Phase portrait features:**

- Small oscillations near $(0, 0)$: closed orbits (libration)
- Large energy: trajectories wrap around (rotation)
- Separatrix: boundary between libration and rotation

# Phase Portrait: Simple Pendulum




**Features:**

- Stable center at $\theta = 0$
- Saddle points at $\theta = \pm\pi$
- Separatrix (homoclinic orbit)
- Libration (oscillation)
- Rotation (full swing)

**Energy:**

$$E = \frac{1}{2}\dot{\theta}^2 - \cos\theta$$

Separatrix: $E = 1$ (just enough to reach top)

# Example 3: Van der Pol Oscillator

**Equation:**

$$\ddot{x} - \mu(1 - x^2)\dot{x} + x = 0 \tag{12}$$

**Key feature:** Nonlinear damping

- $|x| < 1$: Negative damping (energy pumped in)
- $|x| > 1$: Positive damping (energy dissipated)

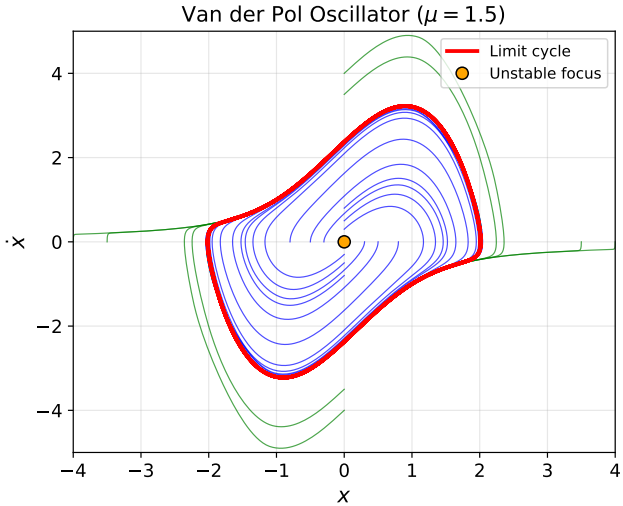
**Result:** Self-sustained oscillations — **limit cycle**

**Behavior:**

- All trajectories (except origin) converge to the limit cycle
- Amplitude determined by system, not initial conditions
- Used to model heartbeats, electrical circuits, predator-prey cycles

**Parameter $\mu$:** Controls "relaxation" — larger $\mu$ gives sharper oscillations

# Phase Portrait: Van der Pol Oscillator




**Features:**

- Origin: **Unstable focus**
- — Stable **limit cycle**
- — Inner trajectories spiral out
- — Outer trajectories spiral in

**Key property:**
All trajectories (except origin) converge to the **same** periodic orbit.

Amplitude determined by **system**, not initial conditions.

# Example 4: Lotka-Volterra (Predator-Prey)

**Model:**

$$\dot{x} = \alpha x - \beta xy \quad \text{(prey growth - predation)} \tag{13}$$

$$\dot{y} = \delta xy - \gamma y \quad \text{(predator growth - death)} \tag{14}$$

**Parameters:** $\alpha, \beta, \gamma, \delta > 0$

**Fixed points:**

- $(0, 0)$ — extinction (saddle, unstable)
- $(\gamma/\delta, \alpha/\beta)$ — coexistence (center)

**Behavior:** Closed orbits around coexistence point

- Populations oscillate periodically
- Predator lags behind prey (phase shift)
- Amplitude depends on initial conditions (not structurally stable)

# Chaotic Dynamical Systems

# Strange Attractors: Gateway to Chaos

**What makes an attractor "strange"?**

**Regular attractors:**

- Fixed point: system stops
- Limit cycle: repeats exactly
- Torus: quasi-periodic, predictable
- Integer dimension (0, 1, 2, ...)

**Long-term behavior:** Predictable

**Strange attractors:**

- Trajectory never repeats
- Bounded but aperiodic
- **Fractal dimension** (e.g., 2.06)
- Sensitive to initial conditions

**Long-term behavior:** Unpredictable!

Fixed Point → Limit Cycle → Torus → **Strange Attractor**

Increasing complexity ⟶

**Next:** The Lorenz system — the most famous strange attractor

# What is Chaos?

**Chaos:** Deterministic but unpredictable long-term behavior

**Key characteristics:**

1. **Sensitivity to initial conditions:** Tiny differences grow exponentially
2. **Deterministic:** No randomness — future determined by present
3. **Bounded:** Trajectories stay in a finite region
4. **Aperiodic:** Never exactly repeats

**"Butterfly effect":** A butterfly flapping its wings in Brazil could cause a tornado in Texas (Lorenz, 1972)

**Practical implication:** Long-term prediction is impossible despite knowing the exact equations!

# Lyapunov Exponents

**Quantifying sensitivity:** How fast do nearby trajectories diverge?
Consider two trajectories starting at $\boldsymbol{x}_0$ and $\boldsymbol{x}_0 + \delta\boldsymbol{x}_0$:

$$\|\delta\boldsymbol{x}(t)\| \approx \|\delta\boldsymbol{x}_0\| e^{\lambda t} \tag{15}$$

**Lyapunov exponent $\lambda$:**

$$\lambda = \lim_{t\to\infty} \lim_{\|\delta\boldsymbol{x}_0\|\to 0} \frac{1}{t} \ln \frac{\|\delta\boldsymbol{x}(t)\|}{\|\delta\boldsymbol{x}_0\|} \tag{16}$$

**Interpretation:**

- $\lambda > 0$: Chaotic (exponential divergence)
- $\lambda = 0$: Marginally stable (e.g., limit cycle)
- $\lambda < 0$: Stable (trajectories converge, strong stability)

**Spectrum:** An $n$-dimensional system has $n$ Lyapunov exponents $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$

# The Lorenz System: Historical Context

**Edward Lorenz (1963):** Meteorologist studying atmospheric convection
**Original model:** 12 ODEs for weather prediction
**Discovery:** Re-running simulation with rounded initial conditions gave completely different results!

**Simplified model:** Truncated Fourier expansion of convection equations

- Rayleigh-Bénard convection: fluid heated from below
- Three modes capture essential dynamics
- Birth of "deterministic chaos" as a field

**Impact:** Fundamentally changed our understanding of predictability in science

# The Lorenz 63 Equations

$$\begin{aligned} \dot{x} &= \sigma(y - x) \\ \dot{y} &= x(\rho - z) - y \\ \dot{z} &= xy - \beta z \end{aligned} \tag{17}$$

**State variables:**

- $x$ — intensity of convective motion
- $y$ — temperature difference (horizontal)
- $z$ — temperature difference (vertical)

**Parameters (classical values):**

- $\sigma = 10$ (Prandtl number)
- $\rho = 28$ (Rayleigh number, normalized)
- $\beta = 8/3$ (geometry factor)

# Lorenz System: Fixed Points

**Setting $\dot{x} = \dot{y} = \dot{z} = 0$:**

**Fixed point 1:** Origin $C_0 = (0, 0, 0)$

- No convection (conduction only)
- Unstable for $\rho > 1$

**Fixed points 2 & 3:** For $\rho > 1$:

$$C_{\pm} = \left(\pm\sqrt{\beta(\rho - 1)}, \pm\sqrt{\beta(\rho - 1)}, \rho - 1\right) \tag{18}$$

- Symmetric pair (convection rolls, opposite directions)
- Stable for $1 < \rho < \rho_H \approx 24.74$
- Unstable for $\rho > \rho_H$ (Hopf bifurcation)

**At $\rho = 28$:** All three fixed points are unstable!

# The Lorenz Attractor

**For $\rho = 28$:** Trajectories are attracted to a **strange attractor**

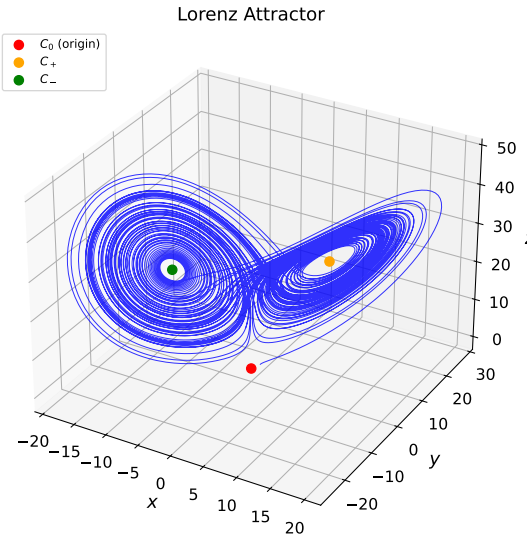
**Properties:**

- **Bounded:** All trajectories eventually enter a bounded region
- **Fractal structure:** Dimension $\approx 2.06$ (not an integer!)
- **Two "wings":** Trajectory switches between lobes unpredictably
- **Zero volume:** Dissipative system contracts phase space

**Lyapunov exponents:** $\lambda_1 \approx 0.91$, $\lambda_2 \approx 0$, $\lambda_3 \approx -14.57$

- $\lambda_1 > 0$: Chaotic!
- $\lambda_2 = 0$: Motion along trajectory
- $\lambda_3 < 0$: Strong contraction onto attractor

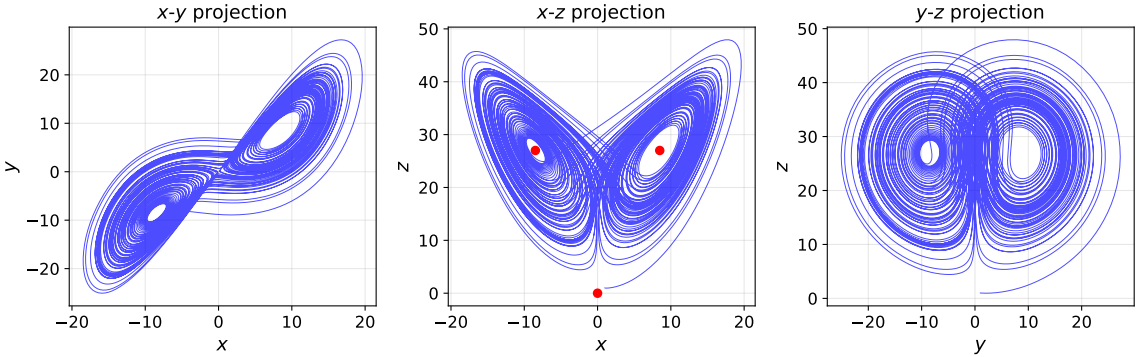
# Lorenz Attractor: Numerical Simulation




**Characteristic behavior:**

- Spirals around one fixed point for a while
- Suddenly switches to the other wing
- Number of loops before switching: unpredictable!

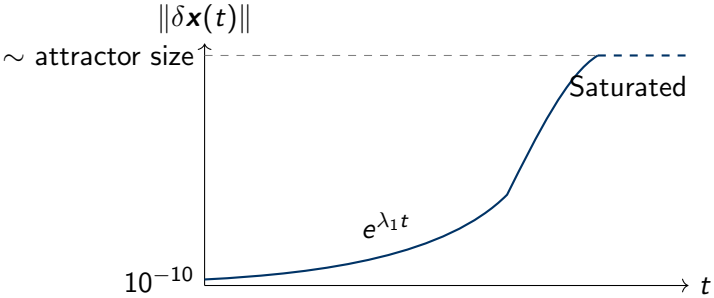
# Lorenz Attractor: 2D Projections




The "butterfly" shape is most visible in the $x$-$z$ projection (middle).

# Sensitivity to Initial Conditions

**Demonstration:** Two trajectories with $\|\delta \boldsymbol{x}_0\| = 10^{-10}$

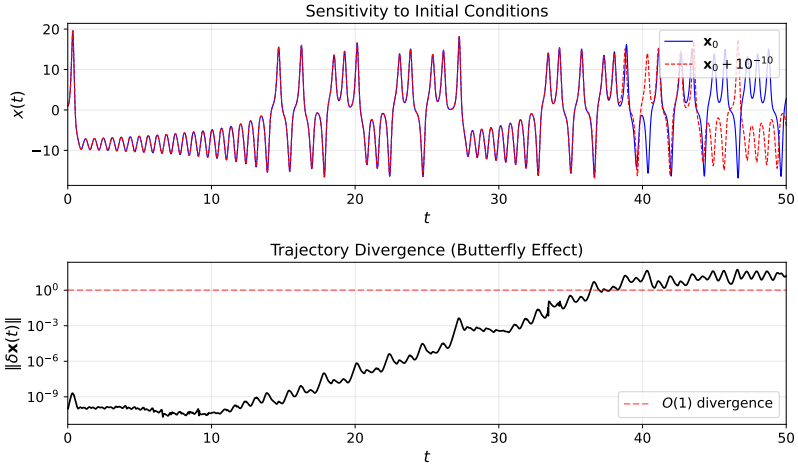



**Prediction horizon:** Time until $\|\delta \boldsymbol{x}(t)\| \sim \mathcal{O}(1)$

$$T_{\text{predict}} \approx \frac{1}{\lambda_1} \ln \frac{1}{\|\delta \boldsymbol{x}_0\|} \tag{19}$$

For Lorenz: $T_{\text{predict}} \approx \frac{1}{0.91} \ln(10^{10}) \approx 25$ time units

# Lorenz System: Trajectory Divergence (Simulation)




Two trajectories starting with $\|\delta \boldsymbol{x}_0\| = 10^{-10}$ remain close until $t \approx 25$, then diverge exponentially. This is the **butterfly effect**.

# Why Chaos Matters for Machine Learning

**Challenges:**

- Small errors in learned dynamics $\Rightarrow$ wrong long-term predictions
- Training on short trajectories may miss attractor structure
- Extrapolation is fundamentally limited by chaos

**Opportunities:**

- **Short-term prediction:** ML can outperform numerical weather prediction
- **Statistical properties:** Learn invariant measure, Lyapunov exponents
- **Attractor reconstruction:** Capture the geometry of strange attractors
- **Hybrid methods:** ML + physics for better forecasts

**Key insight:** Don't aim for perfect long-term trajectories; aim for correct statistics!

# Other Chaotic Systems

**Rössler system:**

$$\dot{x} = -y - z, \quad \dot{y} = x + ay, \quad \dot{z} = b + z(x - c) \tag{20}$$

Simpler than Lorenz, single-scroll attractor

**Double pendulum:**

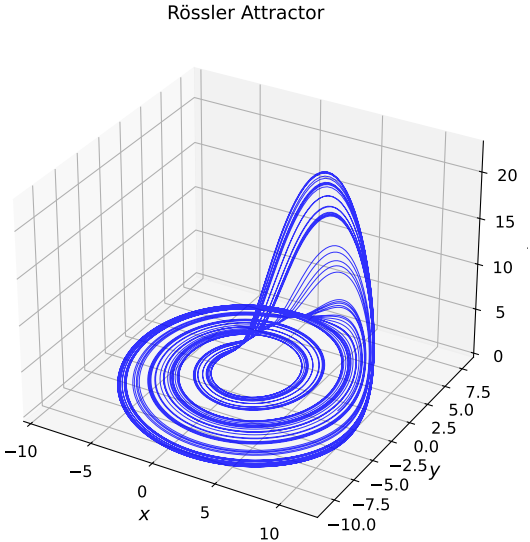
- Two coupled pendulums
- Chaotic for sufficient energy
- Hamiltonian (energy conserving)

**Three-body problem:**

- Three masses under mutual gravitation
- Generally chaotic (no closed-form solution)
- Hamiltonian system

**Turbulence:** The "unsolved problem" — high-dimensional chaos in fluids

# Rössler Attractor: Numerical Simulation




**Rössler system:**

$$\dot{x} = -y - z$$
$$\dot{y} = x + ay$$
$$\dot{z} = b + z(x - c)$$

Parameters: $a = 0.2$, $b = 0.2$, $c = 5.7$

**Features:**

- Single-scroll attractor
- Simpler than Lorenz
- Also chaotic

# Hamiltonian Dynamical Systems

# Why Hamiltonian Systems?

**Many physical systems conserve energy:**

- Planetary motion and celestial mechanics
- Molecular dynamics and quantum mechanics
- Rigid body dynamics and robotics
- Charged particles in electromagnetic fields

**Special structure provides:**

- **Conservation laws:** Energy, momentum, angular momentum
- **Geometric properties:** Symplecticity, volume preservation
- **Long-time stability:** No spurious energy drift
- **Rich mathematical theory:** Canonical transformations, integrability

**For ML:** Exploiting this structure leads to better models!

# Lagrangian Mechanics

**Problem with Newton's approach:**

- $\boldsymbol{F} = m\boldsymbol{a}$ requires Cartesian coordinates $(x, y, z)$
- Constraints (ropes, rails, pivots) create complicated constraint forces
- Must solve for forces we don't actually care about!

**Lagrangian approach:**

- Use coordinates natural to the problem
- Constraints are built into the coordinates
- Work with **energy** (scalar) instead of **forces** (vectors)

$$\boxed{\text{Newton: } \boldsymbol{F} = m\boldsymbol{a}} \xrightarrow[\text{smarter coordinates}]{\text{Same physics,}} \boxed{\text{Lagrange: } \frac{d}{dt}\frac{\partial L}{\partial \dot{q}} = \frac{\partial L}{\partial q}}$$

# What are Generalized Coordinates?

**Definition:** Any set of independent variables $\boldsymbol{q} = (q_1, q_2, \ldots, q_n)$ that completely describe the system's configuration.

**Key properties:**

- **Minimal:** Use exactly as many coordinates as degrees of freedom
- **Independent:** Each $q_i$ can vary independently
- **Complete:** Fully specify the system's position

**Degrees of freedom (DOF):**

$$n = (\text{number of particles}) \times (\text{dimensions}) - (\text{constraints})$$

**Generalized coordinates are NOT necessarily positions!**
They can be angles, arc lengths, or any convenient parameters.

# Generalized Coordinates: Examples

1. **Free particle in 3D**
   - DOF: $3 \times 1 - 0 = 3$
   - Cartesian: $q = (x, y, z)$
   - Spherical: $q = (r, \theta, \phi)$
2. **Simple pendulum**
   - DOF: $2 - 1 = 1$ (fixed length)
   - Bad choice: $(x, y)$ with constraint
   - Good choice: $q = \theta$
3. **Double pendulum**
   - DOF: $4 - 2 = 2$
   - Good choice: $q = (\theta_1, \theta_2)$
4. **Bead on a wire**
   - DOF: $3 - 2 = 1$
   - Good choice: $q = s$ (arc length)
5. **Mass on spring**
   - DOF: $1 - 0 = 1$
   - Good choice: $q = x$ (displacement)
6. **Rigid body rotation**
   - DOF: 3 (rotational)
   - Good choice: Euler angles $(\phi, \theta, \psi)$

**Rule of thumb:** Choose coordinates that make constraints disappear!

# Kinetic and Potential Energy

**Kinetic Energy $T$:** Energy of motion $T = \frac{1}{2}mv^2 = \frac{1}{2}m|\dot{\boldsymbol{r}}|^2$

- Always $\geq 0$ (zero only when at rest)
- Depends on velocity $\dot{\boldsymbol{q}}$ (and possibly $\boldsymbol{q}$)
- For rotation: $T = \frac{1}{2}I\omega^2$

**Potential Energy $V$:** Energy of position (stored energy) $V = V(\boldsymbol{q})$

- Depends only on position, not velocity
- Gravity: $V = mgh$ (height above reference)
- Spring: $V = \frac{1}{2}kx^2$ (displacement from equilibrium)

**Physical meaning:**

- $T$: "How fast is it moving?"
- $V$: "How much work can gravity/spring do on it?"

# The Lagrangian Function

**Definition:**

$$\boxed{L(\boldsymbol{q}, \dot{\boldsymbol{q}}, t) = T - V = \text{Kinetic Energy} - \text{Potential Energy}}$$

**Why $T - V$ and not $T + V$?**

- $T + V = E$ is the **total energy** (conserved if no friction)
- $T - V = L$ is the **Lagrangian** (gives equations of motion)
- The minus sign comes from the *principle of least action*

**The Lagrangian encodes everything:**

- The system's geometry (through $T$)
- The forces acting on it (through $V$)
- From $L$, we can derive the equations of motion!

*"Give me the Lagrangian, and I'll give you the physics." – Richard Feynman*

# The Euler-Lagrange Equation

**The Euler-Lagrange equation:**

$$\boxed{\frac{d}{dt}\frac{\partial L}{\partial \dot{q}_i} - \frac{\partial L}{\partial q_i} = 0, \quad i = 1, \ldots, n}$$

$$\text{Lagrangian } L \xrightarrow{\text{Euler-Lagrange}} \text{Equations of Motion}$$

**Why is this useful?**

1. **Systematic:** Same recipe for ANY mechanical system
2. **Coordinate-free:** Works in any generalized coordinates
3. **No constraint forces:** They disappear automatically!
4. **Reveals structure:** Conservation laws emerge naturally

**Key insight:** Euler-Lagrange = Newton's $F = ma$ in disguise!

# Euler-Lagrange is Newton's Law in Disguise

**For a simple 1D system with $q = x$:** $L = T - V = \frac{1}{2}m\dot{x}^2 - V(x)$
**Compute the Euler-Lagrange terms:**

$$\frac{\partial L}{\partial \dot{x}} = m\dot{x} \quad \text{(momentum)}$$

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{x}} = m\ddot{x} \quad \text{(mass} \times \text{acceleration)}$$

$$\frac{\partial L}{\partial x} = -\frac{\partial V}{\partial x} = F \quad \text{(force)}$$

**Euler-Lagrange equation:**

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{x}} - \frac{\partial L}{\partial x} = 0 \implies m\ddot{x} - F = 0$$

$$\boxed{m\ddot{x} = F = -\frac{\partial V}{\partial x}}$$

This IS Newton's second law! Force = negative gradient of potential.

# Lagrangian Mechanics: Setup

**Generalized coordinates:** $\boldsymbol{q} = (q_1, \ldots, q_n)$ describe configuration

- Not necessarily Cartesian!
- Examples: angles, arc lengths, relative positions

**Lagrangian:** $L(\boldsymbol{q}, \dot{\boldsymbol{q}}, t) = T - V$

- $T(\boldsymbol{q}, \dot{\boldsymbol{q}})$ — kinetic energy
- $V(\boldsymbol{q})$ — potential energy

**Euler-Lagrange equations:**

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{q}_i} - \frac{\partial L}{\partial q_i} = 0, \quad i = 1, \ldots, n \tag{21}$$

**Interpretation:** These are exactly Newton's equations in generalized coordinates!

# Example: Lagrangian of Simple Pendulum

**Generalized coordinate:** $q = \theta$ (angle from vertical)

**Kinetic energy:** $T = \frac{1}{2}m(L\dot{\theta})^2 = \frac{1}{2}mL^2\dot{\theta}^2$
**Potential energy:** $V = -mgL\cos\theta$ (zero at horizontal)
**Lagrangian:**

$$L = \frac{1}{2}mL^2\dot{\theta}^2 + mgL\cos\theta \tag{22}$$

**Euler-Lagrange equation:**

$$\frac{\partial L}{\partial \dot{\theta}} = mL^2\dot{\theta}, \quad \frac{d}{dt}\frac{\partial L}{\partial \dot{\theta}} = mL^2\ddot{\theta} \tag{23}$$

$$\frac{\partial L}{\partial \theta} = -mgL\sin\theta \tag{24}$$

$$\Rightarrow \quad mL^2\ddot{\theta} + mgL\sin\theta = 0 \quad \Rightarrow \quad \ddot{\theta} + \frac{g}{L}\sin\theta = 0 \quad \checkmark \tag{25}$$

# Example: Lagrangian of Simple Pendulum

**Physical interpretation of each term:**

$$\underbrace{\ddot{\theta}}_{\text{angular acceleration}} + \underbrace{\frac{g}{L}}_{\text{natural frequency}^2} \underbrace{\sin\theta}_{\text{restoring factor}} = 0$$

- Angular acceleration $\ddot{\theta}$ is proportional to $-\sin\theta$
- Negative sign: acceleration opposes displacement (restoring!)
- Larger $g$ or smaller $L \Rightarrow$ faster oscillations

**Key insights from the equation:**

1. **Fixed points:** $\sin\theta = 0 \Rightarrow \theta = 0, \pi$ (equilibria)
2. **Nonlinear:** The $\sin\theta$ makes this harder than harmonic oscillator
3. **Small angle:** If $\theta \ll 1$, then $\sin\theta \approx \theta$, giving $\ddot{\theta} + \frac{g}{L}\theta = 0$ (simple harmonic motion with period $T = 2\pi\sqrt{L/g}$)
4. **No friction:** Energy is conserved (no $\dot{\theta}$ term)!

# Lagrangian Mechanics

Choose generalized coordinates $q_1, \ldots, q_n$

↓

Write kinetic energy $T(\boldsymbol{q}, \dot{\boldsymbol{q}})$

↓

Write potential energy $V(\boldsymbol{q})$

↓

Form Lagrangian $L = T - V$

↓

Apply $\frac{d}{dt}\frac{\partial L}{\partial \dot{q}_i} - \frac{\partial L}{\partial q_i} = 0$

↓

Get equations of motion!

**This same recipe works for:** pendulums, springs, rotating bodies, coupled oscillators, celestial mechanics, field theory, quantum mechanics...

# From Lagrangian to Hamiltonian

**Conjugate momentum:** Define momentum conjugate to $q_i$:

$$p_i = \frac{\partial L}{\partial \dot{q}_i} \tag{26}$$

**Legendre transformation:** Transform from $(q, \dot{q})$ to $(q, p)$

$$H(\boldsymbol{q}, \boldsymbol{p}, t) = \sum_{i=1}^{n} p_i \dot{q}_i - L(\boldsymbol{q}, \dot{\boldsymbol{q}}, t) \tag{27}$$

**Physical interpretation:**

- For "natural" systems: $H = T + V$ (total energy!)
- Momentum $p$ and position $q$ are now independent variables

# From Lagrangian to Hamiltonian

**We already have Lagrangian mechanics. Why another formulation?**

**Lagrangian:** $L(q, \dot{q}, t)$

- Variables: position $q$, velocity $\dot{q}$
- 2nd order ODEs
- Great for deriving equations
- Natural for configuration space

**Hamiltonian:** $H(q, p, t)$

- Variables: position $q$, momentum $p$
- 1st order ODEs (twice as many)
- Great for analysis & geometry
- Natural for phase space

**Hamiltonian mechanics is essential for:**

- **Phase space analysis:** Understanding long-term behavior
- **Conservation laws:** Symmetries $\leftrightarrow$ conserved quantities
- **Numerical methods:** Symplectic integrators preserve structure
- **Quantum mechanics:** The Hamiltonian becomes the energy operator
- **Statistical mechanics:** Phase space volume and entropy

# From Lagrangian to Hamiltonian

**Definition:** The momentum $p_i$ conjugate to coordinate $q_i$:

$$p_i = \frac{\partial L}{\partial \dot{q}_i}$$

**Physical intuition:** $p_i$ measures "how much the system resists changes in $\dot{q}_i$"

**Examples:**

| **System** | **Coordinate** $q$ | **Lagrangian** $L$ | **Momentum** $p = \frac{\partial L}{\partial \dot{q}}$ |
|---|---|---|---|
| Free particle | $x$ | $\frac{1}{2}m\dot{x}^2$ | $m\dot{x}$ (linear momentum) |
| Pendulum | $\theta$ | $\frac{1}{2}mL^2\dot{\theta}^2 + mgL\cos\theta$ | $mL^2\dot{\theta}$ (angular momentum) |
| Rotating body | $\phi$ | $\frac{1}{2}I\dot{\phi}^2$ | $I\dot{\phi}$ (angular momentum) |

**Key insight:** Conjugate momentum is NOT always $m \times$ velocity!

- For angles $\rightarrow$ angular momentum
- For EM fields $\rightarrow$ includes vector potential terms

# From Lagrangian to Hamiltonian

**Goal:** Replace velocity $\dot{q}$ with momentum $p$ as the independent variable.

**Why?** Position and momentum are more "symmetric" - both are independent state variables.

$$\boxed{\begin{array}{c}\text{Lagrangian}\\ L(q,\dot{q},t)\end{array}} \xrightarrow[p=\frac{\partial L}{\partial \dot{q}}]{\text{Legendre transform}} \boxed{\begin{array}{c}\text{Hamiltonian}\\ H(q,p,t)\end{array}}$$

**The transformation formula:**

$$\boxed{H(q,p,t) = \sum_{i=1}^{n} p_i \dot{q}_i - L(q,\dot{q},t)}$$

where $\dot{q}_i$ must be expressed in terms of $p_i$ using $p_i = \frac{\partial L}{\partial \dot{q}_i}$.
**For "natural" systems** (kinetic + potential energy):

$$H = T + V = \text{Total Energy}$$

# Hamilton's Equations

**Hamilton's canonical equations:**

$$\dot{q}_i = \frac{\partial H}{\partial p_i}, \quad \dot{p}_i = -\frac{\partial H}{\partial q_i} \tag{28}$$

**First equation:** $\dot{q} = \frac{\partial H}{\partial p}$

- "Velocity from momentum"
- How position changes
- Usually gives $\dot{q} = p/m$

**Second equation:** $\dot{p} = -\frac{\partial H}{\partial q}$

- "Force from potential"
- How momentum changes
- This IS Newton's law: $\dot{p} = F$

**Example: Harmonic oscillator** with $H = \frac{p^2}{2m} + \frac{1}{2}kx^2$

$$\dot{x} = \frac{\partial H}{\partial p} = \frac{p}{m} \qquad \dot{p} = -\frac{\partial H}{\partial x} = -kx$$

Combining: $\ddot{x} = \dot{p}/m = -kx/m$ ✓ (recovers the familiar equation!)

# Phase Space and Symplectic Structure

**Phase space:** The space of all $(q, p)$ pairs $\rightarrow$ dimension $2n$
**State vector:** $\boldsymbol{z} = (q_1, \ldots, q_n, p_1, \ldots, p_n)^T \in \mathbb{R}^{2n}$
**Hamilton's equations in matrix form:**

$$\dot{\boldsymbol{z}} = \boldsymbol{J}\nabla H(\boldsymbol{z}), \quad \boldsymbol{J} = \begin{pmatrix} \boldsymbol{0} & \boldsymbol{I}_n \\ -\boldsymbol{I}_n & \boldsymbol{0} \end{pmatrix} \tag{29}$$

- $\boldsymbol{J}^T = -\boldsymbol{J}$ (skew-symmetric)
- $\boldsymbol{J}^2 = -\boldsymbol{I}$ (like imaginary unit $i$!)
- $\boldsymbol{J}^{-1} = -\boldsymbol{J} = \boldsymbol{J}^T$
- $\boldsymbol{J}$ rotates gradient of $H$ by $90^\circ$ $\rightarrow$ flow is *along* contours of $H$
- Energy $H$ is conserved: $\frac{dH}{dt} = \nabla H \cdot \dot{\boldsymbol{z}} = \nabla H \cdot \boldsymbol{J}\nabla H = 0$
- Phase space volume is preserved (Liouville's theorem)

$\boldsymbol{J}$ is called the **symplectic matrix** or **Poisson tensor**.

# What Does $\boldsymbol{J}$ Actually Do?

**Key insight:** $\boldsymbol{J}$ rotates vectors by 90 in phase space!

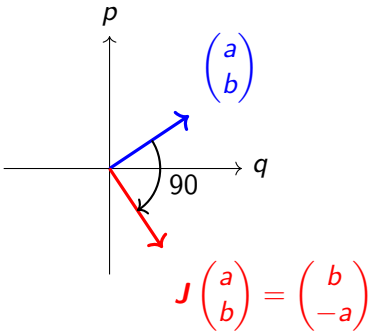
**For $n = 1$:** Let's see what $\boldsymbol{J}$ does to a vector:

$$\boldsymbol{J}\begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}\begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} b \\ -a \end{pmatrix}$$

**Why this matters for Hamilton's equations:**

$$\dot{\boldsymbol{z}} = \boldsymbol{J}\nabla H$$

The flow $\dot{\boldsymbol{z}}$ is perpendicular to $\nabla H$ $\Rightarrow$ trajectories move *along* level curves of $H$!



# Energy Conservation

**Theorem:** For autonomous Hamiltonian systems ($\partial H/\partial t = 0$), energy is conserved.
**Proof:**

$$\frac{dH}{dt} = \sum_i \left( \frac{\partial H}{\partial q_i} \dot{q}_i + \frac{\partial H}{\partial p_i} \dot{p}_i \right) \tag{30}$$

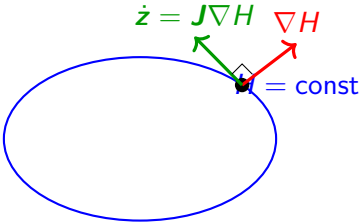
$$= \sum_i \left( \frac{\partial H}{\partial q_i} \frac{\partial H}{\partial p_i} + \frac{\partial H}{\partial p_i} \left( -\frac{\partial H}{\partial q_i} \right) \right) \tag{31}$$

$$= \sum_i \left( \frac{\partial H}{\partial q_i} \frac{\partial H}{\partial p_i} - \frac{\partial H}{\partial p_i} \frac{\partial H}{\partial q_i} \right) = 0 \quad \checkmark \tag{32}$$

**Geometric view:** $\nabla H \perp \dot{\boldsymbol{z}}$, so trajectories follow level sets of $H$!

# Energy Conservation: The Geometry

**Geometric interpretation:**




Flow is tangent to energy surface $\Rightarrow$ energy conserved!

# Symplectic Structure

**Definition:** A map $\boldsymbol{\Phi} : \mathbb{R}^{2n} \to \mathbb{R}^{2n}$ is **symplectic** if:

$$\boldsymbol{D\Phi}^T \boldsymbol{J} \boldsymbol{D\Phi} = \boldsymbol{J} \tag{33}$$

where $\boldsymbol{D\Phi}$ is the Jacobian matrix of $\boldsymbol{\Phi}$.

**Fundamental theorem:** The flow $\phi_t$ of any Hamiltonian system is symplectic!

**Consequences:**

- $\det(\boldsymbol{D}\phi_t) = 1$ (volume preserving — Liouville's theorem)
- Eigenvalues of $\boldsymbol{D}\phi_t$ come in reciprocal pairs
- No asymptotically stable fixed points (only centers and saddles)
- Poincaré recurrence: trajectories return arbitrarily close to start

# Liouville's Theorem

**Statement:** Hamiltonian flow preserves phase space volume.
**Proof:** The divergence of the Hamiltonian vector field is zero:

$$\nabla \cdot \dot{\boldsymbol{z}} = \sum_i \left( \frac{\partial \dot{q}_i}{\partial q_i} + \frac{\partial \dot{p}_i}{\partial p_i} \right) \tag{34}$$

$$= \sum_i \left( \frac{\partial^2 H}{\partial q_i \partial p_i} - \frac{\partial^2 H}{\partial p_i \partial q_i} \right) = 0 \tag{35}$$

**Implication:** If you track a "cloud" of initial conditions, its volume remains constant (though its shape can deform dramatically!).

**Contrast with dissipative systems:** Lorenz system contracts volume ($\nabla \cdot \boldsymbol{f} < 0$), leading to strange attractors with fractal dimension.

# Example: Hamiltonian of Simple Pendulum

**Conjugate momentum:** $p = \frac{\partial L}{\partial \dot{\theta}} = mL^2\dot{\theta}$

So $\dot{\theta} = \frac{p}{mL^2}$

**Hamiltonian:**

$$H = p\dot{\theta} - L = p \cdot \frac{p}{mL^2} - \left(\frac{1}{2}mL^2\dot{\theta}^2 + mgL\cos\theta\right) \tag{36}$$

$$= \frac{p^2}{mL^2} - \frac{p^2}{2mL^2} - mgL\cos\theta \tag{37}$$

$$= \frac{p^2}{2mL^2} - mgL\cos\theta = T + V \tag{38}$$

**Hamilton's equations:**

$$\dot{\theta} = \frac{\partial H}{\partial p} = \frac{p}{mL^2}, \quad \dot{p} = -\frac{\partial H}{\partial \theta} = -mgL\sin\theta \tag{39}$$

# Example: Harmonic Oscillator

**Hamiltonian:**

$$H(q,p) = \frac{p^2}{2m} + \frac{1}{2}kq^2 = \frac{p^2}{2m} + \frac{1}{2}m\omega^2 q^2 \tag{40}$$

**Hamilton's equations:**

$$\dot{q} = \frac{p}{m}, \quad \dot{p} = -m\omega^2 q \tag{41}$$

**Solution:**

$$q(t) = A\cos(\omega t + \phi), \quad p(t) = -mA\omega\sin(\omega t + \phi) \tag{42}$$

**Phase portrait:** Ellipses centered at origin

$$\frac{q^2}{2E/(m\omega^2)} + \frac{p^2}{2mE} = 1 \tag{43}$$

where $E = H$ is the conserved energy.

# Hamilton's Equations via $\boldsymbol{J}$

**Simple harmonic oscillator:** $H = \frac{p^2}{2m} + \frac{1}{2}kq^2$

Compute the gradient of $H$

$$\nabla H = \begin{pmatrix} \frac{\partial H}{\partial q} \\ \frac{\partial H}{\partial p} \end{pmatrix} = \begin{pmatrix} kq \\ p/m \end{pmatrix}$$

Apply $\boldsymbol{J}$

$$\dot{\boldsymbol{z}} = \boldsymbol{J}\nabla H = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} kq \\ p/m \end{pmatrix} = \begin{pmatrix} p/m \\ -kq \end{pmatrix}$$

Read off Hamilton's equations

$$\dot{q} = \frac{p}{m}, \qquad \dot{p} = -kq$$

**Verify these match the standard form:**

$$\dot{q} = \frac{\partial H}{\partial p} = \frac{p}{m} \quad \checkmark \qquad \dot{p} = -\frac{\partial H}{\partial q} = -kq \quad \checkmark$$

# Example: Two-Body (Kepler) Problem

**Setup:** Two masses $m_1, m_2$ with gravitational attraction
**Reduced problem:** In center-of-mass frame, reduce to one body with reduced mass $\mu = \frac{m_1 m_2}{m_1 + m_2}$
**Hamiltonian (polar coordinates):**

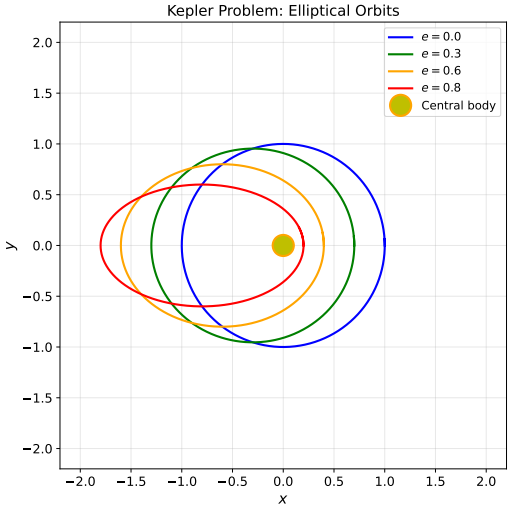
$$H = \frac{p_r^2}{2\mu} + \frac{p_\theta^2}{2\mu r^2} - \frac{G m_1 m_2}{r} \tag{44}$$

**Conserved quantities:**

- Energy $H$ (time translation symmetry)
- Angular momentum $p_\theta = \mu r^2 \dot{\theta}$ (rotation symmetry)

**Orbits:** Conic sections (ellipses, parabolas, hyperbolas)
**This is an integrable system:** 2 DOF, 2 conserved quantities $\Rightarrow$ completely solvable

# Kepler Problem: Elliptical Orbits




**Orbit equation:**

$$r(\theta) = \frac{a(1-e^2)}{1+e\cos\theta}$$

**Eccentricity $e$:**

- $e = 0$: Circle
- $0 < e < 1$: Ellipse
- $e = 1$: Parabola
- $e > 1$: Hyperbola

**Conserved quantities:**

- Energy $E$
- Angular momentum $L$

# Example: Double Pendulum

**Setup:** Two pendulums connected end-to-end
**Coordinates:** $\theta_1, \theta_2$ (angles from vertical)
**Lagrangian:**

$$T = \frac{1}{2}(m_1 + m_2)L_1^2\dot{\theta}_1^2 + \frac{1}{2}m_2 L_2^2\dot{\theta}_2^2 + m_2 L_1 L_2 \dot{\theta}_1\dot{\theta}_2 \cos(\theta_1 - \theta_2) \tag{45}$$

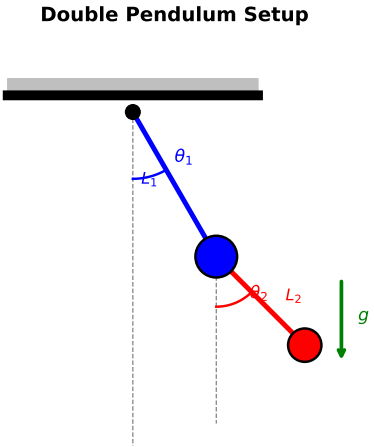
$$V = -(m_1 + m_2)gL_1 \cos\theta_1 - m_2 g L_2 \cos\theta_2 \tag{46}$$

**Properties:**

- **Integrable** at low energies: small oscillations, quasi-periodic
- **Chaotic** at high energies: sensitive dependence on initial conditions
- Energy is conserved (Hamiltonian), but system is chaotic!

**Key point:** Hamiltonian $\neq$ non-chaotic. Energy conservation + chaos can coexist!

# Double Pendulum: Physical Setup




$\theta_1, \theta_2$: angles from vertical

**Configuration:**

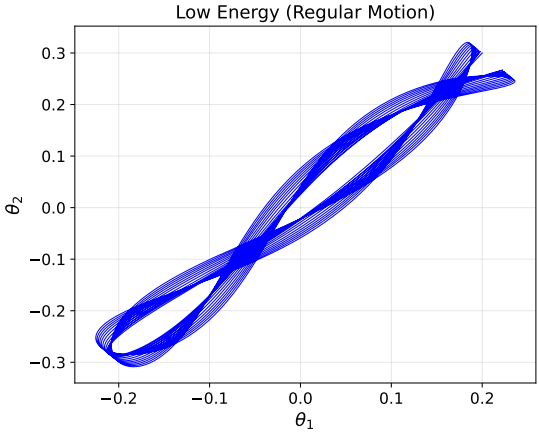
- Two rigid rods of lengths $L_1$, $L_2$
- Point masses $m_1$, $m_2$ at the ends
- First rod pivots at fixed point
- Second rod pivots at $m_1$

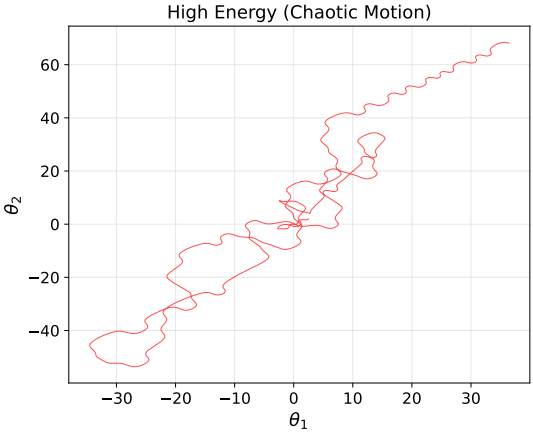
**Generalized coordinates:**

- $\theta_1$: angle of first rod from vertical
- $\theta_2$: angle of second rod from vertical

**Degrees of freedom:** 2 (4D phase space)

# Double Pendulum: Regular vs. Chaotic Motion




**Key point:** Same Hamiltonian system exhibits **both** regular (quasi-periodic) and **chaotic** behavior depending on energy level. Chaos $\neq$ dissipation!

# Separable Hamiltonians

**Definition:** A Hamiltonian is **separable** if:

$$H(\boldsymbol{q}, \boldsymbol{p}) = T(\boldsymbol{p}) + V(\boldsymbol{q}) \tag{47}$$

**Examples:**

- Particle in potential: $H = \frac{\|\boldsymbol{p}\|^2}{2m} + V(\boldsymbol{q})$
- Many-body with pairwise interactions: $H = \sum_i \frac{\|\boldsymbol{p}_i\|^2}{2m_i} + \sum_{i<j} V_{ij}(\|\boldsymbol{q}_i - \boldsymbol{q}_j\|)$

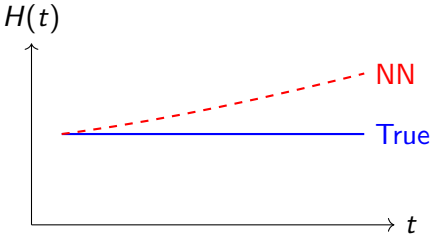
**Why separability matters:**

- Simplifies analysis and numerical methods
- Enables efficient symplectic integrators (splitting methods)
- Natural architecture for neural networks (learn $T$ and $V$ separately)

# Why Structure Matters for Learning

**Standard neural network:** Learn arbitrary $\dot{\boldsymbol{z}} = \hat{\boldsymbol{f}}_\theta(\boldsymbol{z})$

- No guarantee of energy conservation
- May violate symplecticity
- Energy can drift over long simulations




Energy drift breaks long-term predictions

**Solution:** Build Hamiltonian structure into the network architecture!

# Numerical Methods for Simulation

# Why Numerical Methods?

**Goal:** Generate training data by simulating known dynamical systems

**The pipeline:**

1. Define the dynamical system (equations)
2. Choose initial conditions and parameters
3. **Numerically integrate** to get trajectories
4. Use trajectories to train neural networks

**Key considerations:**

- **Accuracy:** How close is the numerical solution to the true solution?
- **Stability:** Does the method blow up for stiff systems?
- **Structure preservation:** Does it respect energy, symplecticity?
- **Efficiency:** How many function evaluations per step?

# Forward Euler Method

**Simplest approach:** Approximate derivative with forward difference

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta t \cdot \boldsymbol{f}(\boldsymbol{x}_k, t_k) \tag{48}$$

**Derivation:** Taylor expansion $\boldsymbol{x}(t + \Delta t) = \boldsymbol{x}(t) + \Delta t \dot{\boldsymbol{x}}(t) + \mathcal{O}(\Delta t^2)$

**Properties:**

- **Order:** 1 (local error $\sim \Delta t^2$, global error $\sim \Delta t$)
- **Explicit:** Only need $\boldsymbol{f}$ at current point
- **Cost:** 1 function evaluation per step
- **Stability:** Limited — requires small $\Delta t$

**Caution:** Generally too inaccurate for serious simulations!

# Runge-Kutta Methods

**Idea:** Evaluate $\boldsymbol{f}$ at multiple points to get higher accuracy
**Classical RK4:**

$$\boldsymbol{k}_1 = \boldsymbol{f}(\boldsymbol{x}_k, t_k) \tag{49}$$

$$\boldsymbol{k}_2 = \boldsymbol{f}(\boldsymbol{x}_k + \tfrac{\Delta t}{2}\boldsymbol{k}_1, t_k + \tfrac{\Delta t}{2}) \tag{50}$$

$$\boldsymbol{k}_3 = \boldsymbol{f}(\boldsymbol{x}_k + \tfrac{\Delta t}{2}\boldsymbol{k}_2, t_k + \tfrac{\Delta t}{2}) \tag{51}$$

$$\boldsymbol{k}_4 = \boldsymbol{f}(\boldsymbol{x}_k + \Delta t \cdot \boldsymbol{k}_3, t_k + \Delta t) \tag{52}$$

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \frac{\Delta t}{6}(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4) \tag{53}$$

**Properties:**

- **Order:** 4 (global error $\sim \Delta t^4$)
- **Cost:** 4 function evaluations per step
- **Widely used:** Good balance of accuracy and cost

# Stiff Systems and Implicit Methods

**Stiff system:** Has widely separated timescales

- Example: Chemical reactions with fast and slow species
- Explicit methods require $\Delta t$ set by *fastest* timescale
- Very inefficient if you care about slow dynamics!

**Backward Euler (implicit):**

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta t \cdot \boldsymbol{f}(\boldsymbol{x}_{k+1}, t_{k+1}) \tag{54}$$

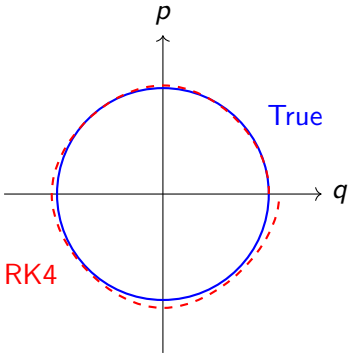
**Properties:**

- Must solve nonlinear equation at each step (Newton iteration)
- **A-stable:** Can use large $\Delta t$ for stiff problems
- More expensive per step, but fewer steps needed

**Adaptive methods:** Adjust $\Delta t$ automatically based on error estimates (e.g., `odeint`, `solve_ivp`)

# Problem: Standard Methods Break Hamiltonian Structure

**Experiment:** Integrate harmonic oscillator with RK4




**Observation:** Energy slowly drifts (usually increases)!
**Why?** RK4 is not symplectic — it doesn't preserve phase space structure.

# Symplectic Integrators

**Goal:** Numerical methods that *exactly* preserve symplectic structure
**Symplectic Euler:** For separable $H = T(p) + V(q)$

$$p_{k+1} = p_k - \Delta t \frac{\partial V}{\partial q}(q_k) \tag{55}$$

$$q_{k+1} = q_k + \Delta t \frac{\partial T}{\partial p}(p_{k+1}) \tag{56}$$

**Note:** Update $p$ first, then use new $p$ to update $q$ (or vice versa)

**Properties:**

- First-order accurate
- Exactly symplectic!
- Energy oscillates but doesn't drift systematically

# Störmer-Verlet (Leapfrog) Method

**Second-order symplectic integrator:**

$$p_{k+1/2} = p_k - \frac{\Delta t}{2}\frac{\partial V}{\partial q}(q_k) \tag{57}$$

$$q_{k+1} = q_k + \Delta t \frac{\partial T}{\partial p}(p_{k+1/2}) \tag{58}$$

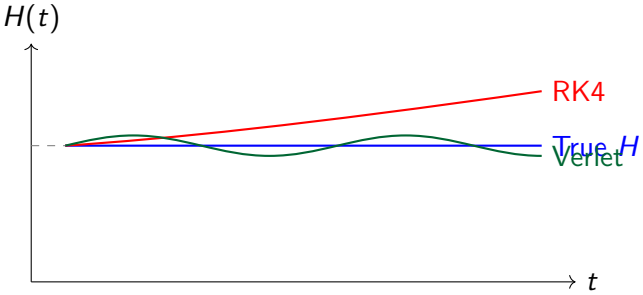
$$p_{k+1} = p_{k+1/2} - \frac{\Delta t}{2}\frac{\partial V}{\partial q}(q_{k+1}) \tag{59}$$

**Properties:**

- **Order:** 2 (same as midpoint rule)
- **Symplectic:** Yes!
- **Time-reversible:** Running backward recovers initial state
- **Cost:** 1 force evaluation per step (if you store $p_{k+1/2}$)

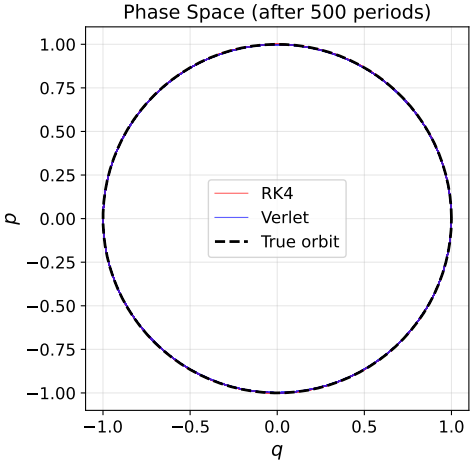
**Widely used:** Molecular dynamics, celestial mechanics, particle physics

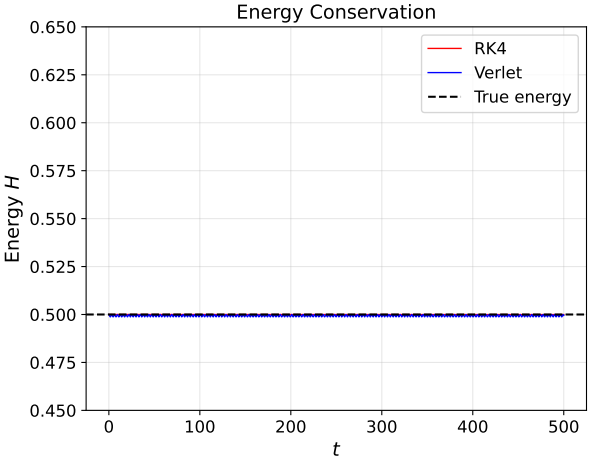
# Energy Behavior: Symplectic vs. Non-Symplectic




**Key insight:** Symplectic methods conserve a *modified* Hamiltonian $\tilde{H} = H + \mathcal{O}(\Delta t^p)$
**Result:** Energy oscillates around true value but never drifts!

# Symplectic vs. Non-Symplectic: Numerical Comparison




Harmonic oscillator integrated for 500 periods with $\Delta t = 0.1$. RK4 shows systematic energy drift and phase space drift; Verlet preserves energy bounds and stays on orbit.

# Higher-Order Symplectic Methods

**Yoshida (4th order):** Composition of Verlet steps with special coefficients

$$\Phi_{\Delta t} = \Phi_{c_1 \Delta t} \circ \Phi_{c_2 \Delta t} \circ \Phi_{c_3 \Delta t} \circ \Phi_{c_2 \Delta t} \circ \Phi_{c_1 \Delta t} \tag{60}$$

where $c_1 = c_4 = \frac{1}{2-2^{1/3}}$, $c_2 = c_3 = \frac{-2^{1/3}}{2-2^{1/3}}$

**Higher orders:** 6th, 8th order methods exist but require more stages

**Trade-off:**

| Method | Order | Force evals/step | Symplectic |
|---|---|---|---|
| Symplectic Euler | 1 | 1 | Yes |
| Störmer-Verlet | 2 | 1 | Yes |
| Yoshida 4 | 4 | 3 | Yes |
| RK4 | 4 | 4 | No |

# Generating Training Data: Best Practices

**For general systems:**

- Use adaptive methods (`solve_ivp` with `RK45` or `DOP853`)
- Set tight tolerances (`rtol=1e-10`, `atol=1e-12`)
- Verify conservation laws if applicable

**For Hamiltonian systems:**

- Prefer symplectic integrators (Verlet, Yoshida)
- Check energy conservation as a diagnostic
- Use fixed time step for consistent data

**Data collection strategy:**

- Multiple trajectories from diverse initial conditions
- Cover the relevant region of phase space
- Include different energy levels / parameter values
- Balance trajectory length vs. number of trajectories

# Python Example: Generating Lorenz Data

**Using SciPy:**

```
import numpy as np
from scipy.integrate import solve_ivp

def lorenz(t, xyz, sigma=10, rho=28, beta=8/3):
x, y, z = xyz
return [sigma*(y-x), x*(rho-z)-y, x*y-beta*z]

t_span = (0, 50)
t_eval = np.linspace(0, 50, 5000)
x0 = [1.0, 1.0, 1.0]

sol = solve_ivp(lorenz, t_span, x0, t_eval=t_eval,
method='RK45', rtol=1e-10, atol=1e-12)

X = sol.y.T   # Shape:  (5000, 3)
```

# Python Example: Symplectic Integration

**Störmer-Verlet for pendulum:**

```
def stormer_verlet(q0, p0, dVdq, dTdp, dt, n_steps):
q, p = q0.copy(), p0.copy()
traj = [(q.copy(), p.copy())]
for _ in range(n_steps):
p_half = p - 0.5*dt*dVdq(q)
q = q + dt*dTdp(p_half)
p = p_half - 0.5*dt*dVdq(q)
traj.append((q.copy(), p.copy()))
return traj

# For pendulum:  dV/dq = g/L * sin(q), dT/dp = p/(mL^2)
dVdq = lambda q:  (g/L) * np.sin(q)
dTdp = lambda p:  p / (m*L**2)
```

# Neural Networks for Predicting Dynamics

# The Learning Problem

**Given:** Trajectory data $\{(\boldsymbol{x}_i, t_i)\}_{i=1}^N$ from numerical simulation
**Goal:** Learn a model that can predict future states

**Two main approaches:**

1. **Learn the vector field:** $\hat{\boldsymbol{f}}_\theta(\boldsymbol{x}) \approx \boldsymbol{f}(\boldsymbol{x})$
   - Then integrate to predict trajectories
   - More interpretable, respects ODE structure
2. **Learn the discrete map:** $\hat{\boldsymbol{x}}_{k+1} = \hat{\boldsymbol{F}}_\theta(\boldsymbol{x}_k)$
   - Direct one-step prediction
   - Simpler to train, but tied to time step $\Delta t$

# Approach 1: Learning the Vector Field

**Architecture:** Neural network $\hat{\boldsymbol{f}}_{\boldsymbol{\theta}} : \mathbb{R}^n \to \mathbb{R}^n$

$$\frac{d\boldsymbol{x}}{dt} = \hat{\boldsymbol{f}}_{\boldsymbol{\theta}}(\boldsymbol{x}) \tag{61}$$

**Training data:** State-derivative pairs $\{(\boldsymbol{x}_i, \dot{\boldsymbol{x}}_i)\}$
**Loss function:**

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \|\dot{\boldsymbol{x}}_i - \hat{\boldsymbol{f}}_{\boldsymbol{\theta}}(\boldsymbol{x}_i)\|_2^2 \tag{62}$$

**Challenge:** Often don't have direct derivative measurements!
**Solutions:**

- Finite differences: $\dot{\boldsymbol{x}}_k \approx \frac{\boldsymbol{x}_{k+1} - \boldsymbol{x}_{k-1}}{2\Delta t}$
- Savitzky-Golay filtering for noisy data
- Trajectory-based loss (requires differentiable ODE solver)

# Approach 2: Learning the Discrete Map

**Architecture:** Neural network $\hat{\boldsymbol{F}}_{\boldsymbol{\theta}} : \mathbb{R}^n \to \mathbb{R}^n$

$$\boldsymbol{x}_{k+1} = \hat{\boldsymbol{F}}_{\boldsymbol{\theta}}(\boldsymbol{x}_k) \tag{63}$$

**Loss function (one-step):**

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{k=1}^{N} \|\boldsymbol{x}_{k+1} - \hat{\boldsymbol{F}}_{\boldsymbol{\theta}}(\boldsymbol{x}_k)\|_2^2 \tag{64}$$

**Multi-step rollout loss:**

$$\mathcal{L}_H(\boldsymbol{\theta}) = \sum_{h=1}^{H} \gamma^{h-1} \|\boldsymbol{x}_{k+h} - \hat{\boldsymbol{F}}_{\boldsymbol{\theta}}^{(h)}(\boldsymbol{x}_k)\|_2^2 \tag{65}$$

where $\hat{\boldsymbol{F}}^{(h)} = \underbrace{\hat{\boldsymbol{F}} \circ \cdots \circ \hat{\boldsymbol{F}}}_{h \text{ times}}$

**Benefit:** Multi-step training reduces error accumulation

# Residual Network Formulation

**Key insight:** ResNet $\approx$ Euler discretization of ODE!
**Residual connection:**

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta t \cdot \hat{\boldsymbol{g}}_{\boldsymbol{\theta}}(\boldsymbol{x}_k) \tag{66}$$

**Compare to Euler:**

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta t \cdot \boldsymbol{f}(\boldsymbol{x}_k) \tag{67}$$

**Benefits:**

- Network only learns the "residual" (change per step)
- Better gradient flow during training
- Natural connection to continuous dynamics
- Can interpret $\hat{\boldsymbol{g}}_{\boldsymbol{\theta}} \approx \boldsymbol{f}$ as learned vector field

**This idea leads to Neural ODEs** (next lecture topic)

# Hamiltonian Neural Networks (HNNs)

**Key idea (Greydanus et al., 2019):** Parameterize the Hamiltonian, not the vector field!

$$\hat{H}_\theta(\boldsymbol{q}, \boldsymbol{p}) = \text{NeuralNetwork}_\theta(\boldsymbol{q}, \boldsymbol{p}) \quad \text{(scalar output)} \tag{68}$$

**Dynamics from structure:**

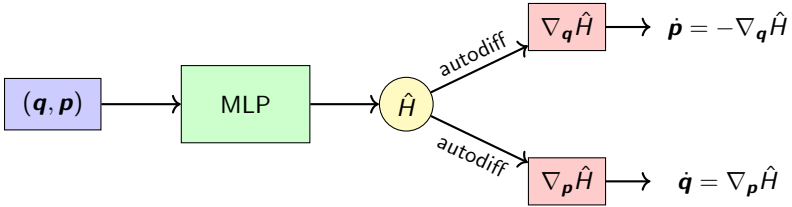
$$\dot{\boldsymbol{q}} = \frac{\partial \hat{H}_\theta}{\partial \boldsymbol{p}}, \quad \dot{\boldsymbol{p}} = -\frac{\partial \hat{H}_\theta}{\partial \boldsymbol{q}} \tag{69}$$

**Training loss:**

$$\mathcal{L} = \sum_i \left\| \dot{\boldsymbol{q}}_i - \left.\frac{\partial \hat{H}_\theta}{\partial \boldsymbol{p}}\right|_i \right\|^2 + \left\| \dot{\boldsymbol{p}}_i + \left.\frac{\partial \hat{H}_\theta}{\partial \boldsymbol{q}}\right|_i \right\|^2 \tag{70}$$

**Result:** Energy conservation is *guaranteed* by construction!

# HNN Architecture




**Implementation:**

- Network outputs scalar $\hat{H}$
- Use automatic differentiation to compute $\nabla_{\boldsymbol{q}} \hat{H}$, $\nabla_{\boldsymbol{p}} \hat{H}$
- Hamilton's equations give the dynamics

# Lagrangian Neural Networks (LNNs)

**Alternative:** Parameterize the Lagrangian instead

$$\hat{L}_\theta(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \text{NeuralNetwork}_\theta(\boldsymbol{q}, \dot{\boldsymbol{q}}) \tag{71}$$

**Euler-Lagrange equations:**

$$\frac{d}{dt}\frac{\partial \hat{L}}{\partial \dot{\boldsymbol{q}}} = \frac{\partial \hat{L}}{\partial \boldsymbol{q}} \tag{72}$$

**Solving for acceleration:**

$$\ddot{\boldsymbol{q}} = \left(\frac{\partial^2 \hat{L}}{\partial \dot{\boldsymbol{q}}^2}\right)^{-1} \left(\frac{\partial \hat{L}}{\partial \boldsymbol{q}} - \frac{\partial^2 \hat{L}}{\partial \boldsymbol{q} \partial \dot{\boldsymbol{q}}}\dot{\boldsymbol{q}}\right) \tag{73}$$

**Advantage:** Works directly with $(\boldsymbol{q}, \dot{\boldsymbol{q}})$ — often more natural
**Disadvantage:** Requires computing Hessian and matrix inverse

# Symplectic Neural Networks (SympNets)

**Idea:** Make the neural network map itself symplectic!
**Approach:** Compose symplectic building blocks

$$\boldsymbol{\Phi}_\theta = \boldsymbol{\Phi}_L \circ \boldsymbol{\Phi}_{L-1} \circ \cdots \circ \boldsymbol{\Phi}_1 \tag{74}$$

**Symplectic layer types:**

- **Linear:** $\begin{pmatrix} \boldsymbol{q}' \\ \boldsymbol{p}' \end{pmatrix} = \begin{pmatrix} \boldsymbol{A} & \boldsymbol{B} \\ \boldsymbol{C} & \boldsymbol{D} \end{pmatrix} \begin{pmatrix} \boldsymbol{q} \\ \boldsymbol{p} \end{pmatrix}$ with $\boldsymbol{A}^T \boldsymbol{C} = \boldsymbol{C}^T \boldsymbol{A}$, etc.
- **Shear (activation):** $\boldsymbol{q}' = \boldsymbol{q} + \nabla_{\boldsymbol{p}} S(\boldsymbol{p})$, $\boldsymbol{p}' = \boldsymbol{p}$

**Result:** Symplecticity by construction — no gradients of $H$ needed!

# Comparison of Approaches

| Method | Energy | Symplectic | Coordinates | Cost |
|---|---|---|---|---|
| Standard NN | No | No | Any | Low |
| HNN | Yes | Yes* | $(\boldsymbol{q}, \boldsymbol{p})$ | Medium |
| LNN | Yes | Yes* | $(\boldsymbol{q}, \dot{\boldsymbol{q}})$ | High |
| SympNet | No** | Yes | $(\boldsymbol{q}, \boldsymbol{p})$ | Medium |

* With appropriate symplectic integrator
** Energy approximately conserved due to symplecticity

**Guidelines:**

- Hamiltonian system with $(q, p)$ data $\rightarrow$ HNN or SympNet
- Hamiltonian system with $(q, \dot{q})$ data $\rightarrow$ LNN
- Non-Hamiltonian system $\rightarrow$ Standard NN or Neural ODE

# Practical Considerations

**Architecture choices:**

- Depth: 3-5 hidden layers typically sufficient
- Width: 64-256 neurons per layer
- Activation: Tanh or Swish (smooth, for differentiation)

**Training tips:**

- Normalize inputs and outputs
- Use learning rate scheduling
- Multi-step rollout loss for long-term accuracy
- Validate on held-out trajectories *and* new initial conditions

**Evaluation:**

- Short-term: MSE on test trajectories
- Long-term: Energy drift, attractor shape, Lyapunov exponents
- Generalization: Performance on unseen initial conditions

# Summary

# Key Takeaways

1. **Dynamical systems** are described by ODEs; key concepts include fixed points, stability, and attractors
2. **Chaotic systems** (Lorenz) are deterministic but unpredictable long-term; Lyapunov exponents quantify this
3. **Hamiltonian systems** have special structure: energy conservation, symplecticity, volume preservation
4. **Numerical methods** matter: symplectic integrators preserve structure that standard methods destroy
5. **Neural networks** can learn dynamics; structure-preserving architectures (HNN, LNN, SympNet) outperform generic NNs for Hamiltonian systems

# Next Topic: Neural ODEs

**Coming up:** Neural Ordinary Differential Equations

**Key ideas:**

- Continuous-depth neural networks
- Adjoint sensitivity method for efficient gradients
- Connection between ResNets and ODEs
- Normalizing flows and density estimation

**Building on today:**

- Vector field learning $\rightarrow$ Neural ODE
- Differentiating through ODE solvers
- Combining with Hamiltonian structure

# References


- Strogatz, S. H. (2015). *Nonlinear Dynamics and Chaos*. Westview Press.
- Lorenz, E. N. (1963). Deterministic nonperiodic flow. *J. Atmospheric Sciences*.
- Hairer, E., Lubich, C., & Wanner, G. (2006). *Geometric Numerical Integration*. Springer.
- Greydanus, S., Dzamba, M., & Yosinski, J. (2019). Hamiltonian Neural Networks. *NeurIPS*.
- Cranmer, M., et al. (2020). Lagrangian Neural Networks. *ICLR Workshop*.
- Jin, P., Zhang, Z., Zhu, A., Tang, Y., & Karniadakis, G. E. (2020). SympNets. *NeurIPS*.
- Chen, R. T. Q., et al. (2018). Neural Ordinary Differential Equations. *NeurIPS*.

# Questions?